

BRAINHACK 2026

RoboVerse: Flight Challenge
Search and Map

AGENDA

Main goal: To provide teams an idea to develop obstacle avoidance capabilities with localization.

Drone to use: x500_vision

1. Localisation using Visual Odometry
1. Obstacle Avoidance Using Depth Camera
1. Using Depth Data

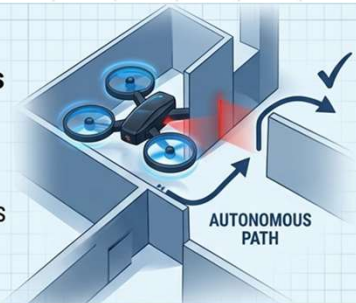
Localisation using Visual Odometry

The need to estimate position

- Your code might have enabled the quadcopter to go around the environment and react to obstacles autonomously.
- The quadcopter keep going around in circles aimlessly, and missing unexplored areas.
- If the quadcopter is able to estimate its location, your code is able to estimate if an area has been explored or unexplored.
- We cannot generate a global map without a position and orientation of the drone.

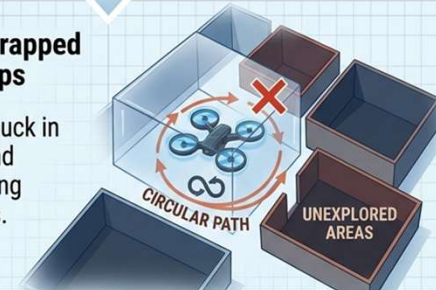
ACHIEVEMENT: Successful Autonomous Navigation

- The drone moves freely and avoids obstacles, following an 'autonomous path' with a checkmark.



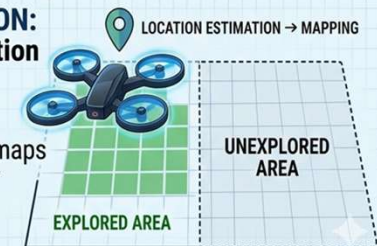
LIMITATION: Entrapped in Repetitive Loops

- The drone gets stuck in 'circular paths' and misses surrounding unexplored zones.



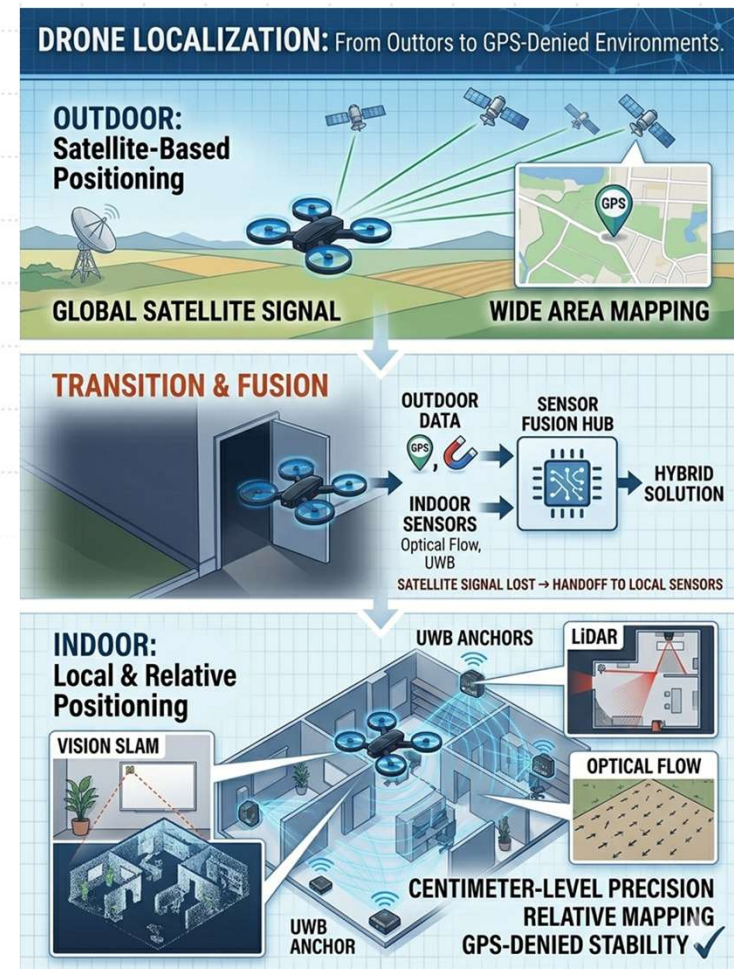
POTENTIAL SOLUTION: Location and Exploration Mapping

- By using 'location estimation', the drone maps and col 'explored areas' to discover new zones.



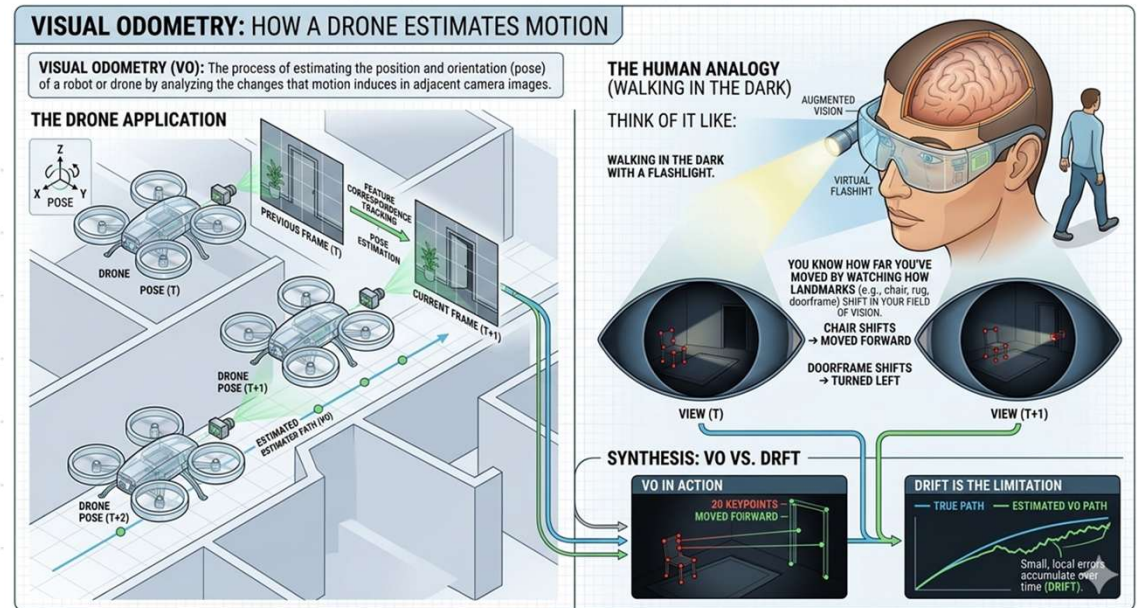
Estimate position

- The primary challenge of localization is the environment:
 - Outdoor flights usually rely on Global Navigation Satellite System (GNSS) signals but these can be easily denied.
 - Indoor flights require "GNSS-denied" techniques that use local environmental features.
- Since satellite signals cannot penetrate roofs or walls or can be disrupted, drones must use "onboard" or "local" references.
- An example of onboard reference is the Inertial Measurement Unit (IMU) on a drone.
- Local reference examples are:
 - LIDAR
 - UWB
 - Vision-based such as optical flow, visual odometry, stereo camera.



Visual Odometry

- Visual Odometry (VO) is the process of estimating the position and orientation (pose) by analyzing the changes that motion induces in adjacent camera images.
 - Adjacent images can be frames after frames or stereo camera images.
- Think of it like a human walking in the dark with a flashlight: you know how far you've moved by watching how landmarks (a chair, a rug, a doorframe) shift in your field of vision.
- Pros: Easily deployable



How it works (high level pipeline)

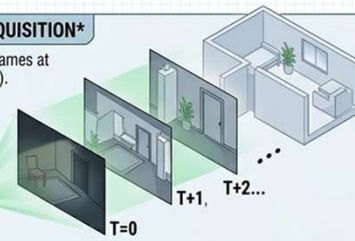
- Step 1. Image Acquisition: The camera captures frames at a high rate (e.g., 30 fps).
- Step 2. Feature Detection: The software identifies "keypoints" in the image—sharp corners, edges, or unique textures.
- Step 3. Feature Tracking/Matching: The system looks for those same keypoints in the next frame.
- Step 4. Motion Estimation: By measuring the geometric displacement of these points, the system calculates the camera's movement (X, Y, Z and rotation).
- Step 5. Local Optimization: It constantly "polishes" the path to reduce small calculation errors.

VISUAL ODOMETRY PIPELINE: A FIVE-STEP PROCESS

STEP 1: IMAGE ACQUISITION*

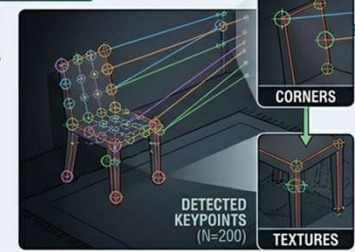
The camera captures frames at a high rate (e.g., 30 fps).

High-speed capture camera (e.g., from <IMAL0).



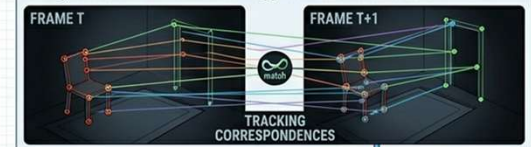
STEP 2: FEATURE DETECTION*

The software identifies "keypoints" in the image—sharp corners, edges, or unique textures.



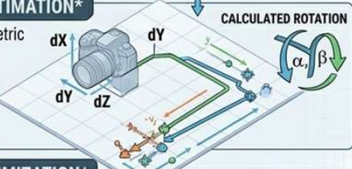
STEP 3: FEATURE TRACKING/MATCHING*

The system looks for those same keypoints in the next frame.



STEP 4: MOTION ESTIMATION*

By measuring the geometric displacement of these points, the system calculates the camera's movement (X, Y, Z and rotation).



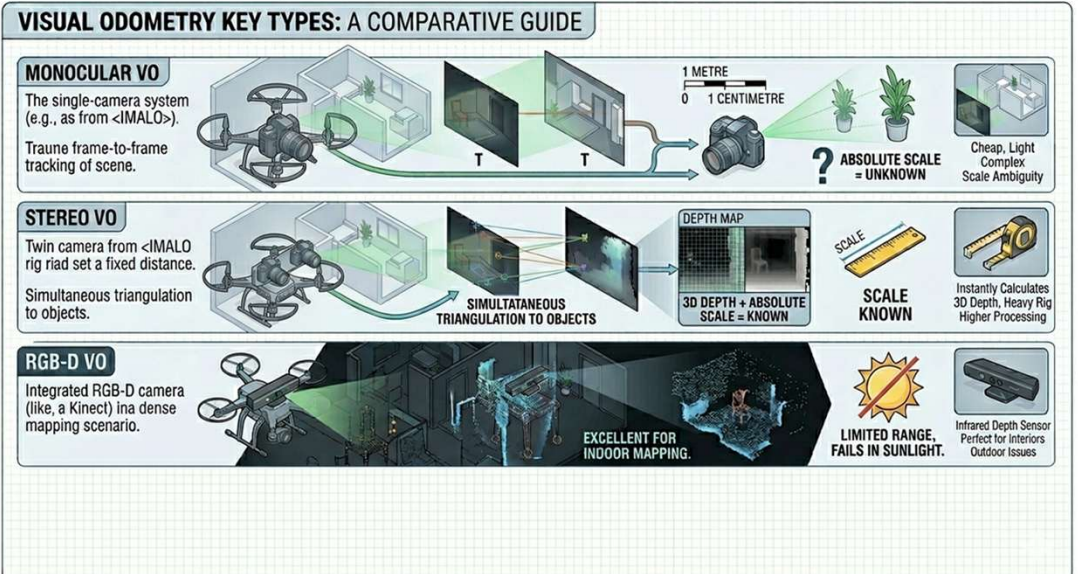
STEP 5: LOCAL OPTIMIZATION*

It constantly "polishes" the path to reduce small calculation errors.



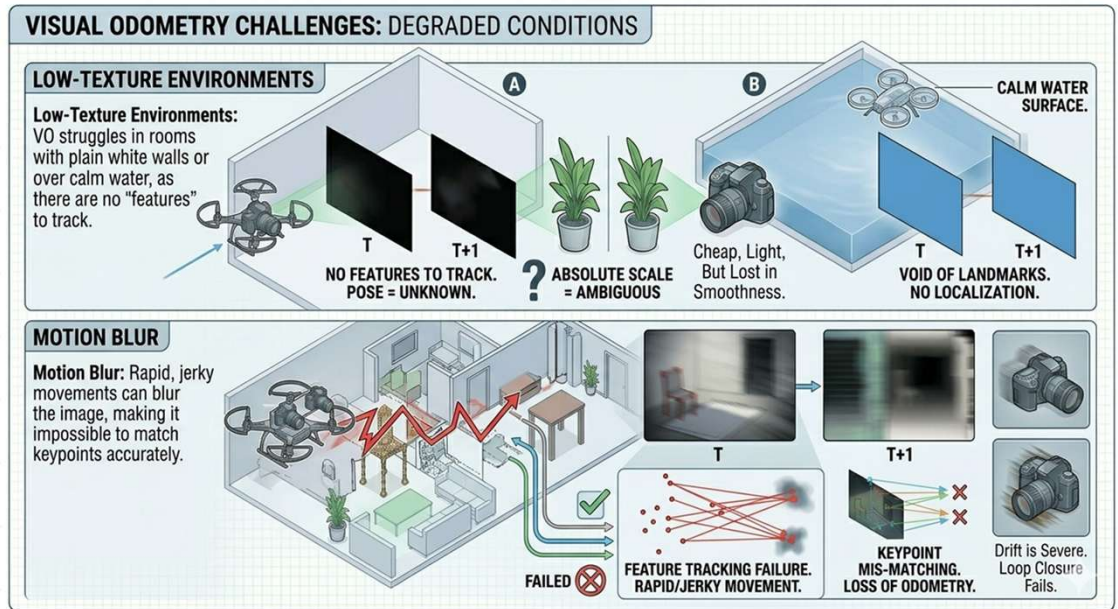
Common types of Visual Odometry

Type	Description	Pros/Cons
Monocular	Uses a single camera.	Pros: Cheap, light. Cons: Cannot determine "absolute scale" (doesn't know if a move is 1 meter or 1 centimeter without extra sensors).
Stereo	Uses two cameras with a known distance between them.	Pros: Instantly calculates 3D depth and absolute scale. Cons: Heavier, higher processing power required.
RGB-D	Uses an infrared depth sensor (like a Kinect).	Pros: Excellent for indoor mapping. Cons: Limited range; fails in bright sunlight.



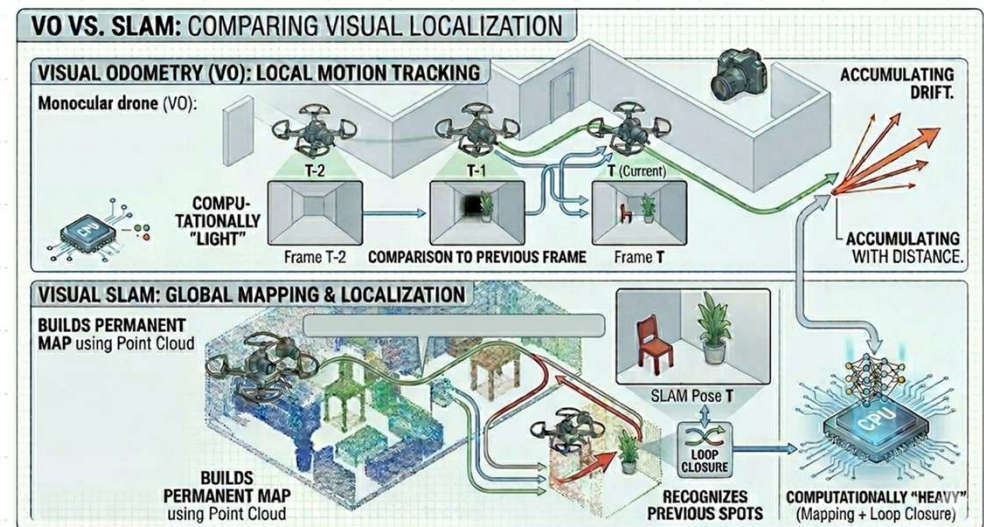
Limitations to Keep in Mind

- The "Drift" Problem: Because VO calculates motion relative to the previous frame, tiny errors add up over time. Without a global map (SLAM) or GPS to correct it, the estimated position will eventually "drift" away from reality.
- Low-Texture Environments: VO struggles in rooms with plain white walls or over calm water, as there are no "features" to track
- Motion Blur: Rapid, jerky movements can blur the image, making it impossible to match keypoints accurately.
- Lighting: Extreme darkness or blinding glare can "blind" the algorithm.



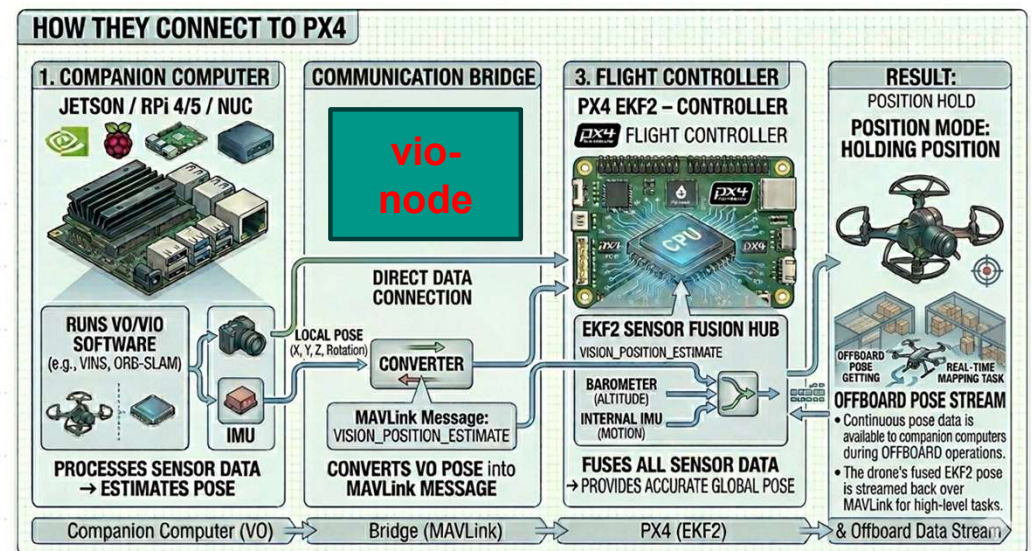
VO vs. SLAM: What's the difference?

- Visual Odometry: Only cares about the local path (where am I compared to 1 second ago?). It is computationally "light".
- Visual SLAM (Simultaneous Localization and Mapping): Builds a permanent map of the whole environment and recognizes when it has returned to a previous spot (Loop Closure) to fix all accumulated drift. It is much more "heavy" on the processor.
- As such, the Qualifier will only require teams to achieve some simple map without visual slam due to Visual SLAM's heavy processor demand.



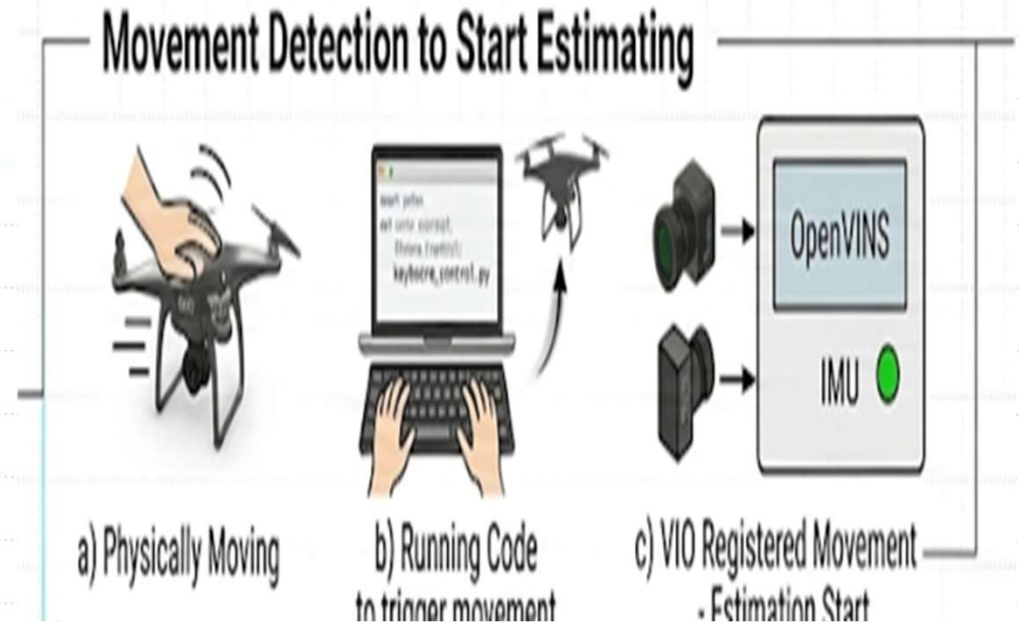
V0 options on PX4 for position estimates

- PX4 Autopilot can use these opensource Visual-Inertial Odometry (VIO) for position estimate.
- VINS-Fusion:
 - It is an extension of VINS-Mono and supports monocular, stereo, and even GPS-aided configurations but require heavier computation power.
- ORB-SLAM3:
 - While technically a SLAM system, ORB-SLAM3 is widely used for its visual odometry capabilities. It is famous for its ability to handle "Loop Closure" (recognizing a place it has been before to zero out drift).
- Kimera (by MIT SPARK Lab):
 - Kimera is a "Metric-Semantic" SLAM library. It was designed specifically for high-speed drone flight but require ROS/ROS2.
- OpenVINS:
 - OpenVINS is often used by researchers who need high-accuracy state estimation with lower CPU overhead and due its modular extensible architecture.



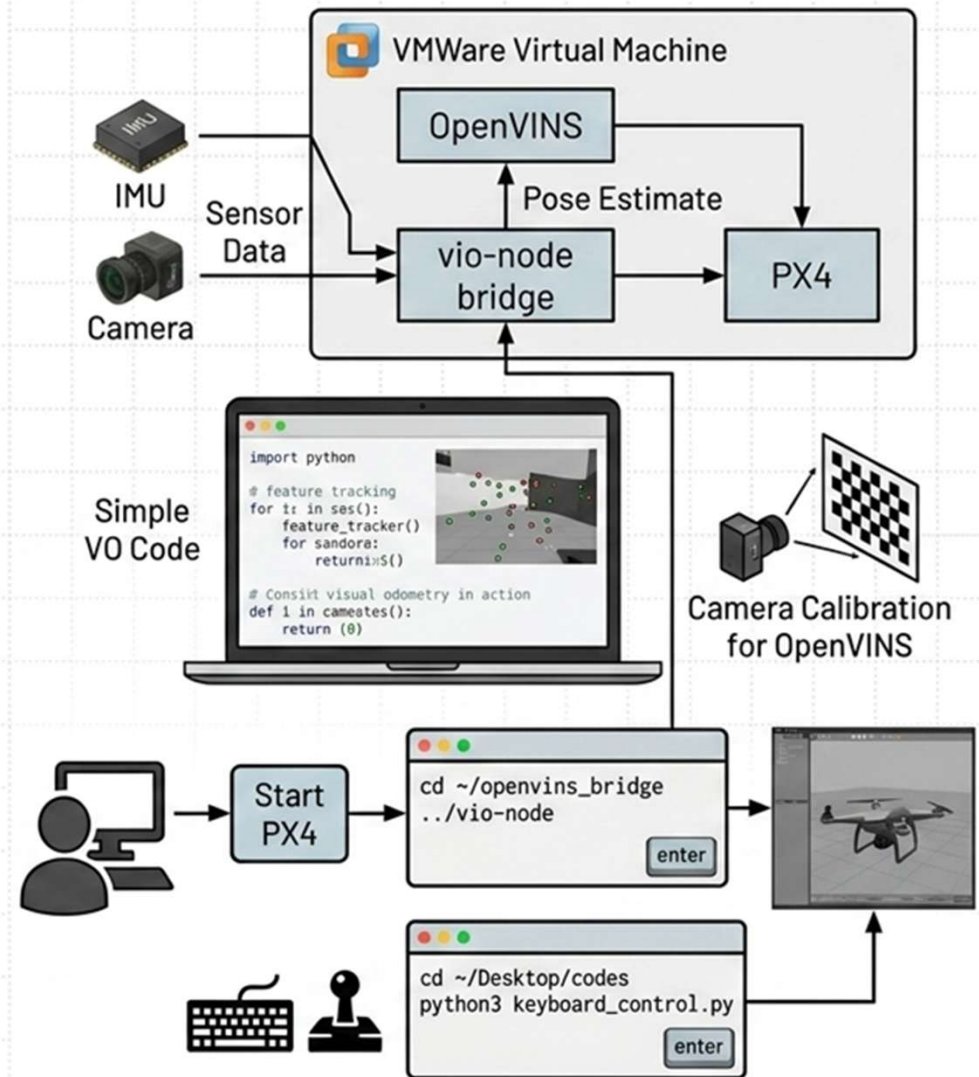
How to use OpenVINS or other VIO with PX4

- OpenVINS or other VIO system replaces GPS to provide PX4 with position estimates.
- Before OpenVINS or other VIO system, VIO system needs to detect movement of drone. One can physically move the drone or run some codes to trigger movement, for the VIO to register movement and start estimating.
- Then VIO can send the position estimate to PX4 or any other drone system.
- PX4 will not allow arming without position estimate as a form of failsafe. To allow arming, the only way is to physically move the drone.
- PX4 has a virtual drone that simulates vision odometry in Gazebo which PX4 allows arming upon startup without the need of triggering the VIO processing. The drone is x500_vision.



Using OpenVINS on the vmware machine

- The VMWare machine has OpenVINS installed, together with vio-node bridge that:
 - Get IMU and camera, and feed into OpenVINS
 - Get pose estimate from OpenVINS and feed into PX4.
- Run the following to try out OpenVINS:
 - Start PX4 as usual which also start the gazebo simulator.
 - START a new Terminal window and enter the following commands:
 1. `cd ~/openvins_bridge`
 2. `./vio-node`
 - Start a new Terminal window and enter the following commands:
 - `cd ~/Desktop/codes`
 - `python3 keyboard_control.py`

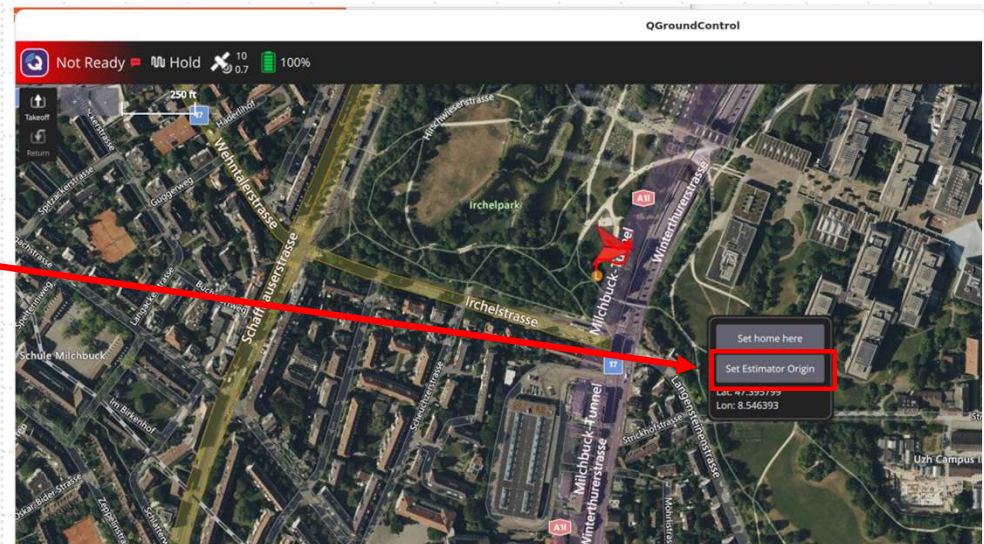
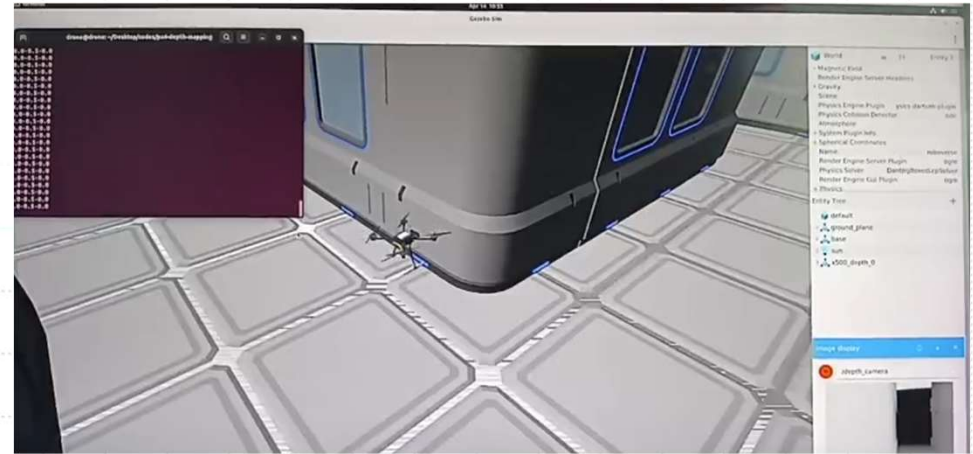


Using PX4 x500_vision for VIO simulation

- A physical drone with VIO capabilities will be used in the Final. The subsequent slide will cover more using VIO, coding some simple VIO and how to setup VIO.
- Qualifier will use PX4 x500_vision instead of OpenVINS for simulation of a VIO capable drone.
- To use PX4 x500_vision, select (2) for vehicle model when you run `./start_px4.sh`
- After PX4 commander ready to accept command, type the following at `px4>`

```
commander set ekf_origin 47.397742  
8.545594 488.0
```

- OR use SET ESTIMATOR ORIGIN using QGroundControl after you click anywhere on the map.
- This to tell PX4 to set the ekf origin so that it starts to fuse the sensor data.



Getting pose using mavsdk

- After PX4's EKF2 fuses VIO from x500_vision or OpenVINS and streams the result via LOCAL_POSITION_NED, which MAVSDK reads seamlessly.
- Your code can get position update using `async for pos_vel in drone.drone.telemetry.position_velocity_ned():`
- The position data is in meters in terms of NED, in other words position of the drone body itself and relative to the takeoff position of the drone (or where your drone get initialized). Refer to [get_position.py](#) for details.
- Your code could call `position_velocity_ned()` as and when in your logic, or create python task and shared state to get position in background while your main loop runs. See [get_position_task.py](#) for details.
 - Within `get_position_with_task.py`, there is a block that says "your control logic starts here". This is where your autonomous code goes.

INSTRUCTOR: to step through key part of `get_position_with_tasks`, especially the creating task and use shared state to pass position data from monitor loop to the main loop.

```
1 # main.py - Integration of Drone class with async position monitoring
2 import asyncio
3 from drone_control import Drone # Your provided class
4 from mavsdk.offboard import VelocityNedYaw
5
6 class SharedState:
7     """Thread-safe(ish) container for inter-task data in a single event loop."""
8     def __init__(self):
9         self.latest_position = None # NED position from telemetry
10        self.latest_yaw = None
11        self.is_armed = False
12        self.control_active = False
13
14    async def position_monitor_task(drone: Drone, state: SharedState, stop_event: asyncio.Event):
15        """
16        Background task streaming NED position and Yaw updates concurrently.
17        """
18        print("Position monitor task started...")
19
20        async def stream_position():
21            async for pos_vel in drone.drone.telemetry.position_velocity_ned():
22                if stop_event.is_set():
23                    break
24                state.latest_position = pos_vel.position
25
26        async def stream_yaw():
27            async for att in drone.drone.telemetry.attitude_euler():
28                if stop_event.is_set():
29                    break
30                state.latest_yaw = att.yaw_deg
31
32        try:
33            # Run both streams concurrently
34            await asyncio.gather(stream_position(), stream_yaw())
35
36        except asyncio.CancelledError:
37            print("✂ Position monitor task cancelled.")
38        except Exception as e:
39            print(f"Monitor error: {type(e).__name__}: {e}")
40
41
42    async def control_loop(drone: Drone, state: SharedState, stop_event: asyncio.Event):
43        """
44        Main offboard control loop using your Drone class methods.
45        Example: simple position-hold with telemetry logging.
46        """
47        print("\nStarting main control loop...")
48
49        # Ensure offboard is active (your class handles this in arm_and_takeoff)
50        await drone.arm_and_takeoff()
51        state.is_armed = True
```

Obstacle using Depth Camera

[Drone to use: x500_vision]

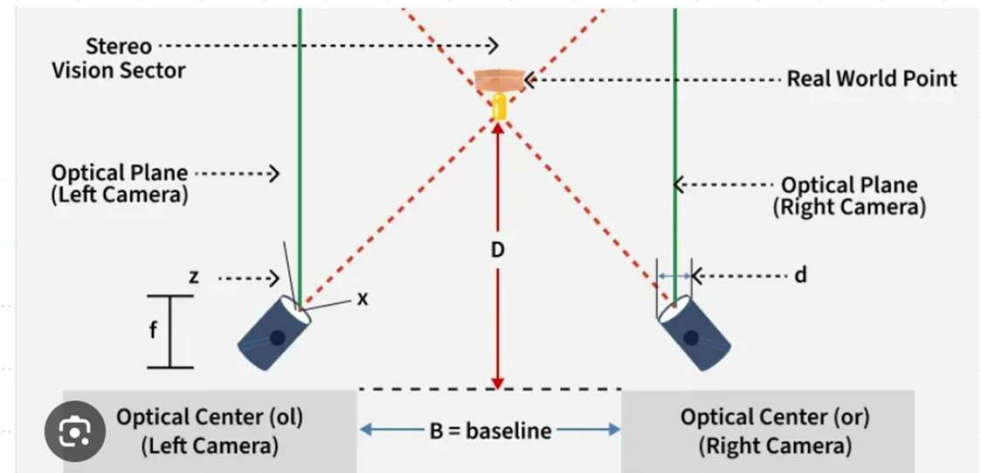
What is a Depth Camera?

- A depth camera (also known as an RGB-D camera) is a type of sensor that provides more than just a 2D color image. It captures spatial data, allowing the system to know exactly how far away every pixel in the frame is from the lens.
- Depth Camera provides color (R, G, B) + distance (D).
- The Result: A 3D "snapshot" of the environment rather than a flat projection.



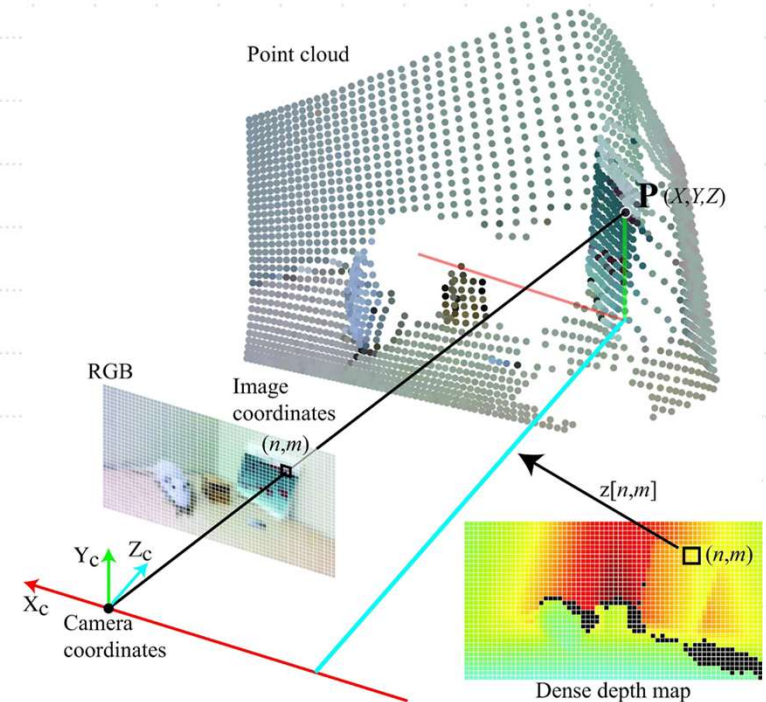
Depth Camera Technologies

- Depth cameras use one of three primary technologies to estimate distance:
 - Stereo Vision: Uses two cameras slightly apart (like human eyes). By comparing the horizontal shift of objects between the two images (disparity), the distance is calculated.
 - Structured Light: Projects a known pattern of infrared dots or lines onto a scene. The camera observes how the pattern deforms on surfaces to calculate depth.
 - Time-of-Flight (ToF): Emits a light pulse and measures the nanoseconds it takes for the light to bounce off an object and return.
 - Depth Estimation can also be done using one camera (Monocular Depth Estimation)



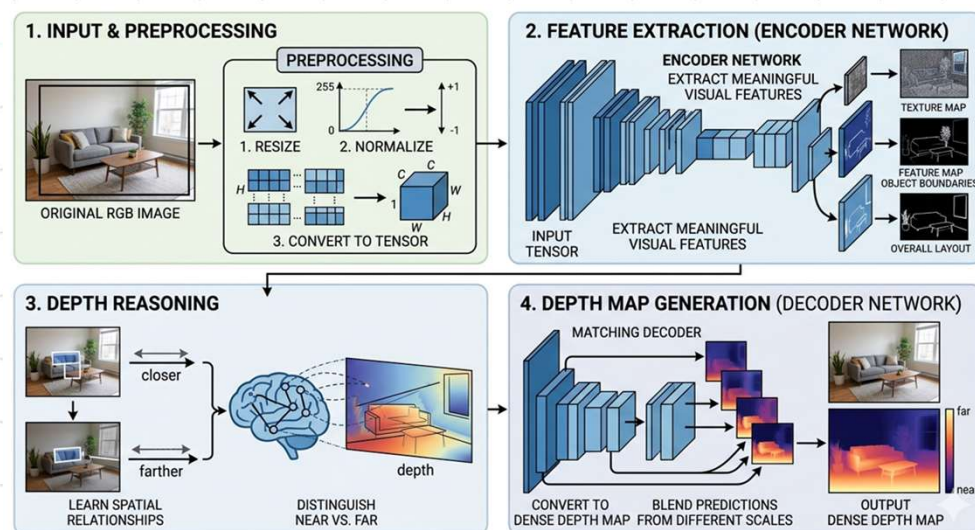
Monocular Depth Estimation

- Depth Estimation can also be done using one camera.
- Often, a single image doesn't contain information at the correct scale.
- So, early monocular depth estimation methods relied on straightforward visual rules tied to camera geometry. Cues like perspective, object size, and whether one object blocked another were used to estimate distance.
- Most modern monocular depth estimation systems use deep learning, which refers to neural networks with many layers that can learn complex patterns from data. These models learn to predict depth directly from images and produce depth maps.



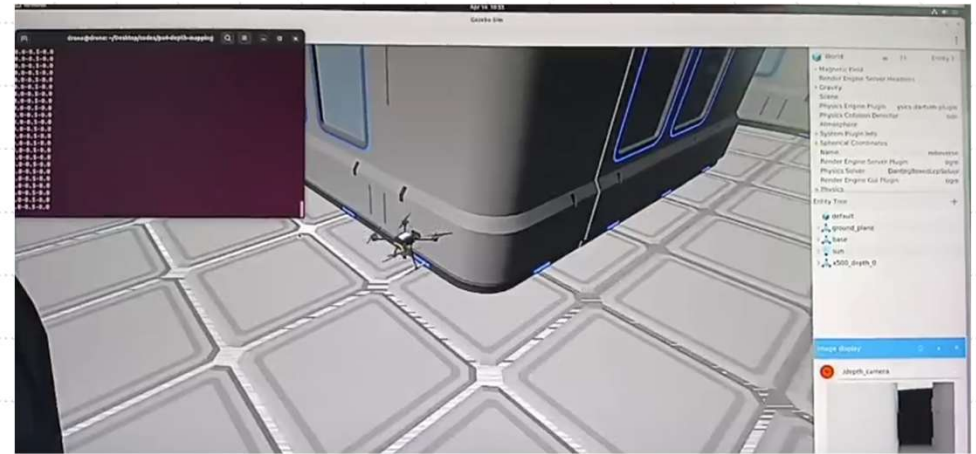
How monocular depth estimation models work

- Most monocular depth estimation models follow a similar process to convert a single image into a depth map:
 - Input and preprocessing: The original image is usually resized, normalized, and converted into a tensor.
 - Feature extraction: An encoder network analyzes the image to extract meaningful visual features such as textures, object boundaries, and the overall layout of the scene.
 - Depth reasoning: At this stage, it learns which regions of the image are closer to the camera and which are farther away.
 - Depth map generation: A decoder then converts this information into a dense depth map. Each pixel in the image is assigned a depth value, often by blending predictions from different scales to improve accuracy and consistency.



Depth Estimation in Qualifier and Final

- For Qualifier, teams are required to use the depth camera in X500 virtual drone. [x500_depth]
- For Final, the physical drones are equipped with RealSense D435i or D455 stereo depth camera.
- Both comes with API that allows team to get depth data easily.
- As such, teams need not develop depth estimation algorithm from scratch.



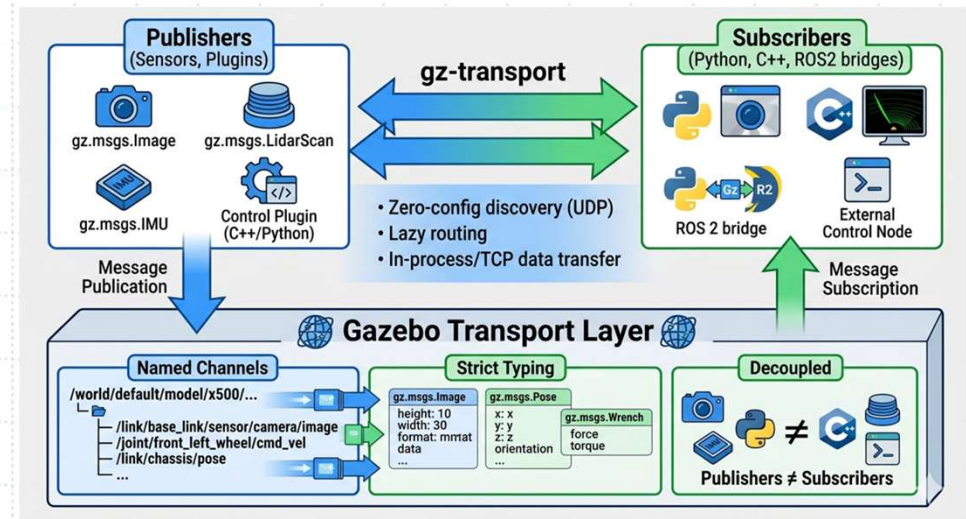
Using depth data provided by Gazebo

[Drone to use: x500_vision]

How gazebo provide sensor data

- PX4 does not provide data on peripherals sensors like camera, stereo camera. Those are provided by Gazebo.
- Gazebo provides data using a middleware library called gz-transport. The library supports many programming languages.
- It allows different parts of Gazebo and its plugins—such as rendering engines, physics plugins, sensors, and external controllers—to exchange data seamlessly without needing detailed knowledge of each other using publisher-subscriber data exchange.
- Data are published via named channel, also known as topic. Subscribe to topics to get data. Some examples are:

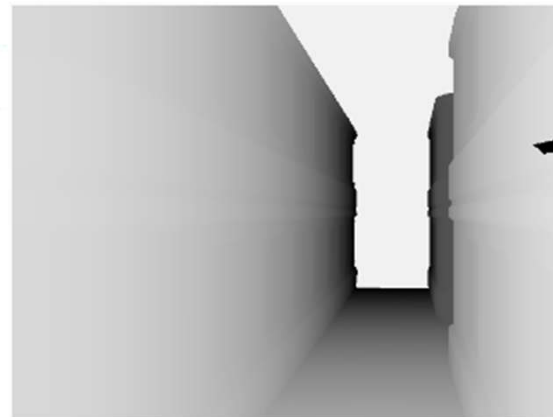
Publisher Device	Named Channel (Topic)
Lidar Sensor	/world/default/model/x500/link/base_link/sensor/lidar/scan
IMU Sensor	/world/default/model/x500/link/base_link/sensor/imu/data
Plugin	/control_plugin/commands



How to discover topics

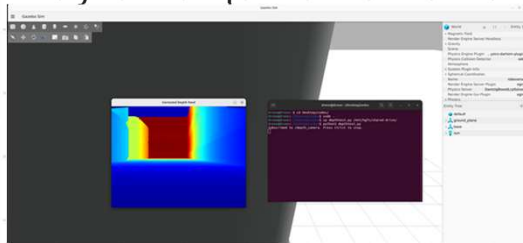
- Use `gz topic` to find out more about topics that are published after gazebo starts.
- Try it out using Terminal after the PX4 simulator is started.
- For example, X500_depth provide both depth data via `/depth_camera` and monocular camera image via another topic.
- First step, it is to know the names of topics and what types of message publish by the topic.
- GZ libraries provides libraries to use those message types.

Command	Purpose	⬇
<code>gz topic -l</code>	List all active topics in running simulation	
<code>gz topic -i -t /topic/name</code>	Show publishers, subscribers, message type	
<code>gz topic -e -t /topic/name</code>	Echo/print raw messages to terminal	
<code>gz topic -p -t /topic -m gz.msgs.Pose -p 'position: {x:1, y:0, z:2}'</code>	Publish test message manually	



How to subscribe to topics using python

- Use gz.transport13 and gz.msgs10 libraries (line 3 and 4).
- After knowing the name of the topic, add to the topic name to subscribe (for e.g. line 31)
- When there is data published onto that topic, gz-transport calls the function that is provided as callback when calling node.subscribe()(for e.g. line 34).
- As such, your code needs to provide a callback function that takes in a parameter to consume the data.
- /depth_camera provides Image type.
- depthtest.py (code on the right) shows how to display visually the depth data using opencv.



depthtest.py

```
1 import numpy as np
2 import cv2
3 from gz.msgs10.image_pb2 import Image
4 from gz.transport13 import Node
5
6 def depth_callback(msg):
7     # 1. Convert raw bytes to float32 (meters)
8     depth_array = np.frombuffer(msg.data, dtype=np.float32)
9     depth_img = depth_array.reshape((msg.height, msg.width))
10
11     # 2. Handle invalid data (NaN or Inf)
12     # Replaces 'too far' or 'too close' errors with 0.0
13     depth_img = np.nan_to_num(depth_img, nan=0.0, posinf=0.0, neginf=0.0)
14
15     # 3. Scale for visibility
16     # This rescales the actual meter values (0-10m) to a 0-255 grayscale range
17     visible_depth = cv2.normalize(depth_img, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
18
19     # 4. (Optional) Apply a Colormap for better intuition
20     # Makes the "black" image look like a heat map (blue=far, red=near)
21     color_depth = cv2.applyColorMap(visible_depth, cv2.COLORMAP_JET)
22
23     cv2.imshow("Corrected Depth Feed", color_depth)
24     cv2.waitKey(1)
25
26 def main():
27     # Initialize the Gazebo Transport Node
28     node = Node()
29
30     # Replace with your actual depth camera topic from 'gz topic -l'
31     topic = "/depth_camera"
32
33     # Subscribe to the topic
34     if node.subscribe(Image, topic, depth_callback):
35         print(f"Subscribed to {topic}. Press Ctrl+C to stop.")
36     else:
37         print(f"Failed to subscribe to {topic}.")
38         return
39
40     # Keep the script running to receive callbacks
41     try:
42         import time
43         while True:
44             time.sleep(1)
45     except KeyboardInterrupt:
46         print("Shutting down...")
47
48 if __name__ == "__main__":
49     main()
```

Understanding gz.msgs.Image Data Structure and displaying it

- Gz-transport provides depth data using Image data structure:

Field	Type	Description
header	Header	Sequence number, timestamp, frame_id
width	uint32	Image width in pixels (e.g., 640)
height	uint32	Image height in pixels (e.g., 480)
step	uint32	Bytes per row (may include memory padding)
format	string	Pixel encoding: "FLOAT32" or "32FC1"
data	bytes	Raw byte buffer containing pixel values

- IMPORTANT: data is not a 2D array. It's a flat byte stream that must be interpreted using width, height and step and format.
- For example, see line 8 and line 9 where numpy uses width and height information.
- depthtest.py (code on the right) shows how to display visually the depth data using opencv.

depthtest.py

```
1 import numpy as np
2 import cv2
3 from gz.msgs10.image_pb2 import Image
4 from gz.transport13 import Node
5
6 def depth_callback(msg):
7     # 1. Convert raw bytes to float32 (meters)
8     depth_array = np.frombuffer(msg.data, dtype=np.float32)
9     depth_img = depth_array.reshape((msg.height, msg.width))
10
11     # 2. Handle invalid data (NaN or Inf)
12     # Replaces 'too far' or 'too close' errors with 0.0
13     depth_img = np.nan_to_num(depth_img, nan=0.0, posinf=0.0, neginf=0.0)
14
15     # 3. Scale for visibility
16     # This rescales the actual meter values (0-10m) to a 0-255 grayscale range
17     visible_depth = cv2.normalize(depth_img, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
18
19     # 4. (Optional) Apply a Colormap for better intuition
20     # Makes the "black" image look like a heat map (blue=far, red=near)
21     color_depth = cv2.applyColorMap(visible_depth, cv2.COLORMAP_JET)
22
23     cv2.imshow("Corrected Depth Feed", color_depth)
24     cv2.waitKey(1)
25
26 def main():
27     # Initialize the Gazebo Transport Node
28     node = Node()
29
30     # Replace with your actual depth camera topic from 'gz topic -l'
31     topic = "/depth_camera"
32
33     # Subscribe to the topic
34     if node.subscribe(Image, topic, depth_callback):
35         print(f"Subscribed to {topic}. Press Ctrl+C to stop.")
36     else:
37         print(f"Failed to subscribe to {topic}.")
38         return
39
40     # Keep the script running to receive callbacks
41     try:
42         import time
43         while True:
44             time.sleep(1)
45     except KeyboardInterrupt:
46         print("Shutting down...")
47
48 if __name__ == "__main__":
49     main()
```


What does float32 mean?

- Each pixel value uses 4 byte to represent distance in meters from camera optical center to surface.
- See line 13 to 21, especially line 13 which tell numpy how to interpret the data.
- Range:
 - 0.0 (invalid/no return) → near_clip → far_clip (e.g., 10.0m)
- Precision:
 - ~1mm accuracy in typical ranges (sufficient for drone navigation)
- Special Values:
 - 0.0 or NaN: No valid depth (ray hit sky, transparent material, or out of range)
 - inf: Clipped at far boundary
- Why not uint16?
 - uint16 requires scaling (e.g., mm = value/1000). float32 stores meters directly, simplifying 3D math and point cloud generation.

depthtest.py

```
1 import numpy as np
2 import cv2
3 from gz.msgs10.image_pb2 import Image
4 from gz.transport13 import Node
5
6 def depth_callback(msg):
7     # 1. Convert raw bytes to float32 (meters)
8     depth_array = np.frombuffer(msg.data, dtype=np.float32)
9     depth_img = depth_array.reshape((msg.height, msg.width))
10
11     # 2. Handle invalid data (NaN or Inf)
12     # Replaces 'too far' or 'too close' errors with 0.0
13     depth_img = np.nan_to_num(depth_img, nan=0.0, posinf=0.0, neginf=0.0)
14
15     # 3. Scale for visibility
16     # This rescales the actual meter values (0-10m) to a 0-255 grayscale range
17     visible_depth = cv2.normalize(depth_img, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
18
19     # 4. (Optional) Apply a Colormap for better intuition
20     # Makes the "black" image look like a heat map (blue=far, red=near)
21     color_depth = cv2.applyColorMap(visible_depth, cv2.COLORMAP_JET)
22
23     cv2.imshow("Corrected Depth Feed", color_depth)
24     cv2.waitKey(1)
25
26 def main():
27     # Initialize the Gazebo Transport Node
28     node = Node()
29
30     # Replace with your actual depth camera topic from 'gz topic -l'
31     topic = "/depth_camera"
32
33     # Subscribe to the topic
34     if node.subscribe(Image, topic, depth_callback):
35         print(f"Subscribed to {topic}. Press Ctrl+C to stop.")
36     else:
37         print(f"Failed to subscribe to {topic}.")
38         return
39
40     # Keep the script running to receive callbacks
41     try:
42         import time
43         while True:
44             time.sleep(1)
45     except KeyboardInterrupt:
46         print("Shutting down...")
47
48 if __name__ == "__main__":
49     main()
```

Using depth data for obstacle avoidance

PLEASE refer to depth_receiver.py while reading this slide

Make the depthtest.py into a depth_receiver class for ease of use

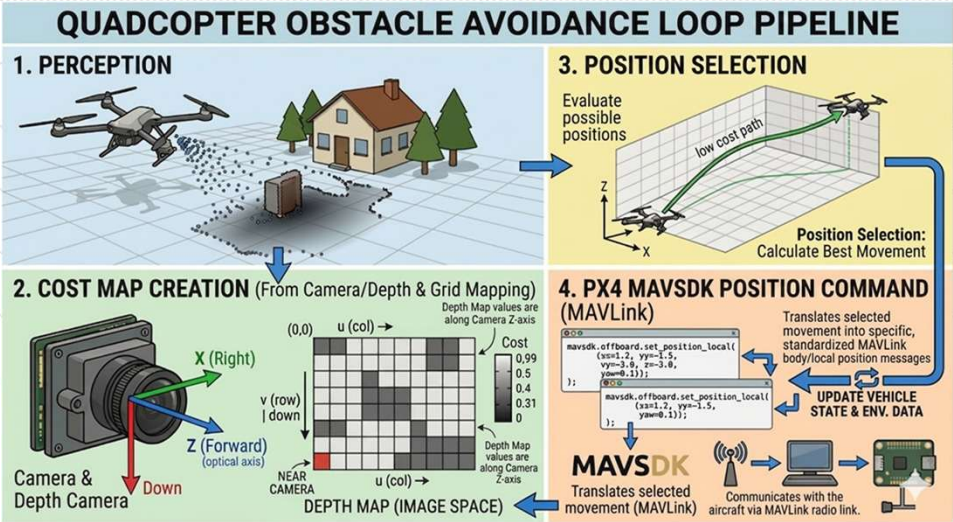
- Let's extract the core part of the depthtest.py that turns Image data into numpy object, and put it into a depth_receiver class.
- The depth_receiver class will have a function, called `get_frame()` that allows other function to get the latest image data numpy object.

```
depth_receiver.py
1  from gz.transport13 import Node
2  from gz.msgs10.image_pb2 import Image
3  import numpy as np
4  import threading
5
6  class DepthReceiver:
7      def __init__(self, topic):
8          self.node = Node()
9          self.depth = None
10         self.lock = threading.Lock()
11
12         self.node.subscribe(Image, topic, self.callback)
13
14     def callback(self, msg: Image):
15         depth = np.frombuffer(msg.data, dtype=np.float32)
16         depth = depth.reshape((msg.height, msg.width))
17
18         with self.lock:
19             self.depth = depth
20
21     def get_frame(self):
22         with self.lock:
23             return None if self.depth is None else self.depth.copy()
```


PLEASE refer to AvoidancePlanner.py and drone_control.py while reading this slide

A closed-loop pipeline for avoiding obstacles using depth camera data

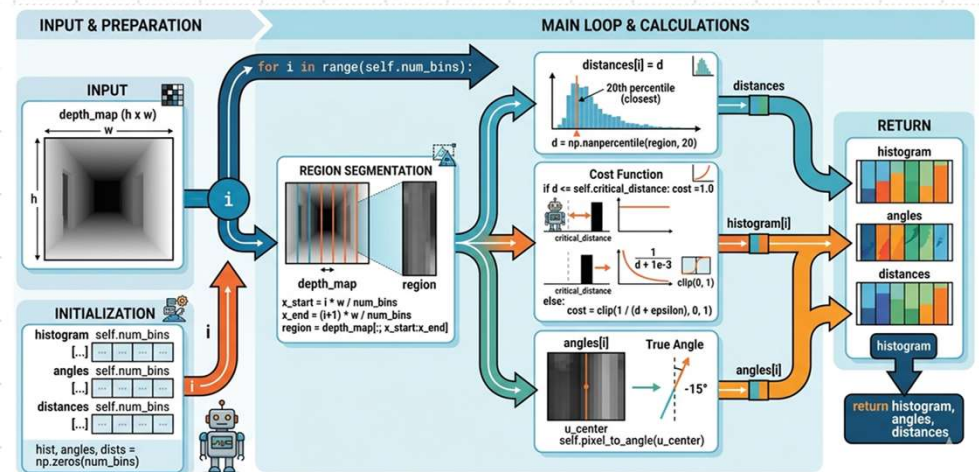
- Let us start by building a pipeline that makes quadcopter reacts to presence of obstacles (like tree, blocks, columns or walls) such that it moves left or right and forward, depending on distance to object.
 - A rather low-level reactive mechanism. Later we will build higher level planning upon this pipeline to have mapping and searching capabilities.
- A closed-loop pipeline that continuously
 - Get data from depth camera.
 - perform using `get_frame()` in `depth_receiver.py`
 - Processing depth map data to derive position (relative to the drone takeoff position) to fly.
 - performed by `compute_position_ned` function in `AvoidancePlannner.py`.
 - Use the `set_position` in `drone_control.py` to control quadcopter
 - Repeat (1) until goal is met



PLEASE refer to AvoidancePlanner.py while reading this slide

Summarizing the depth data

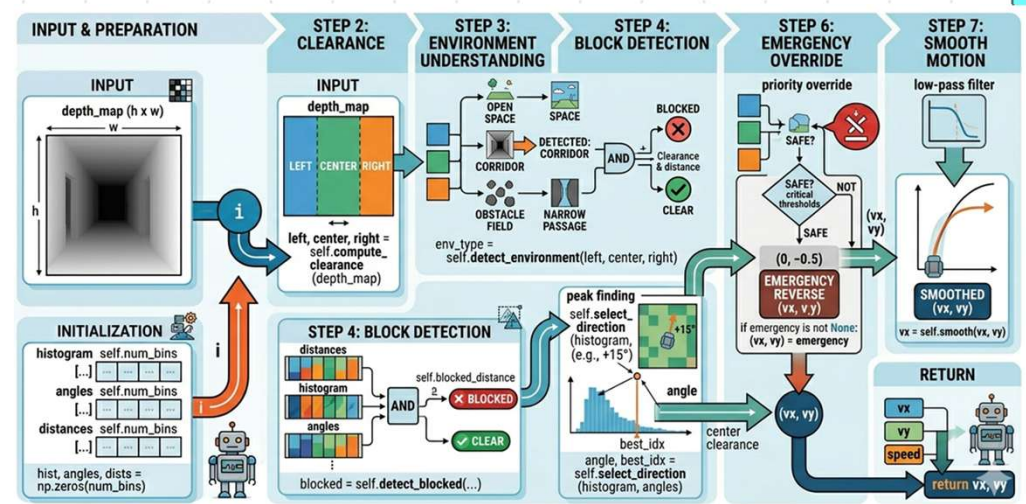
- The depth_receiver.py provides a depth_map, 2D array that describes distance of objects from the stereo camera which is quite challenging if it is not pre-processed or summarized.
- First, segment depth_map into equal region based on `num_of_bins` in the code. This could be something that you could tweak to affect the performance. Then the following is done on distance data in each region:
 - A distance value of that region is determined by calculating 20 percentile of each region, excluding NAN data.
 - Then that distance value is used to get a cost value where distance less than `critical_distance` gets highest cost.
 - Angle of the center of the region is calculated using the camera's intrinsic values (focal length of the camera along x axis and principal point x-coordinate).



PLEASE refer to AvoidancePlanner.py while reading this slide

Using the summarized data to make speed decision

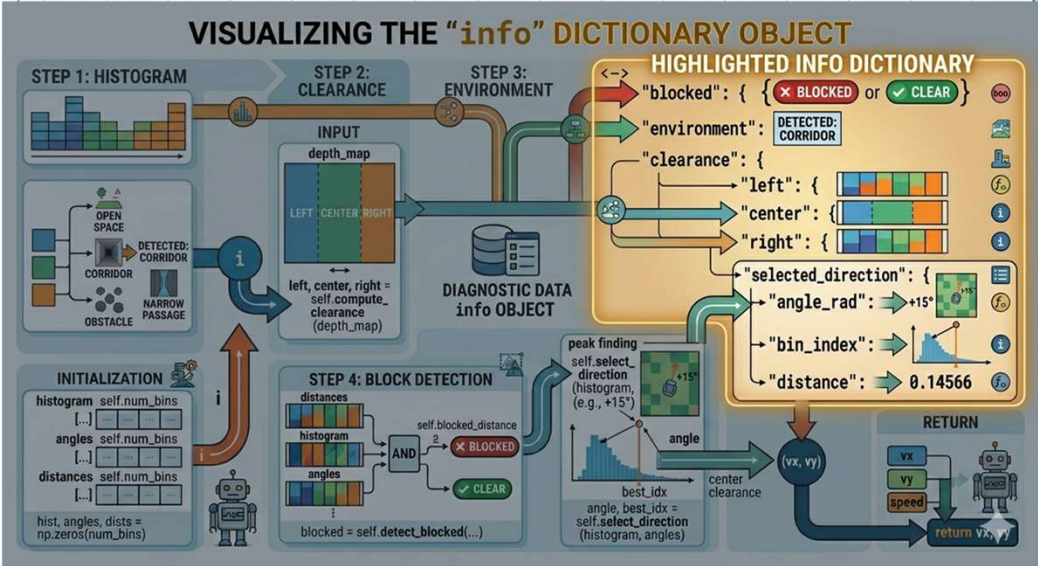
- The summarized data are further computed to derive the velocity to move forward or left/right:
 - The regions bin are further summarized into three distance of left, center, and right.
 - The left, center, right is then assessed to see if it is open space straight ahead, or if it is a narrow passage, or even it is completely blocked, based on the **critical distance** and **safe distance value** values that are used to initialize the VelocityPlanner.
 - If not blocked, the best angle to fly is selected using the histogram, which is to estimate the velocity.
- The velocity is further “smoothed” before provided to the calling function by considering previous velocity and the alpha value. The **alpha** value can be tweaked to affect the performance.
- The processing and summarizing are performed when **compute_position_ned** function in AvoidancePlanner.py is called with a depthmap numpy array.



PLEASE refer to AvoidancePlanner.py while reading this slide

Output from `compute_position_ned` function

- The `compute_position_ned` returns three float –
 - the north position in m
 - the east position in m
 - the down position in m
- Note: The position is relative to where the drone takeoff.
- Also a dictionary object that describes:
 - Blocked: true/false
 - Environment: "open" or "forward_clear" or "left_open" or "right_open"
 - Clearance:
 - "left" – a float value to describe object distance at left region.
 - "center" – a float value to describe object distance at center region.
 - "right" – a float value to describe object distance at the right region.
- Besides the position, the dictionary object are very important for higher level planning such as what to do when the entire front is blocked and etc.



PLEASE refer to avoid.py while reading this slide

Using compute_velocity to implement the pipeline

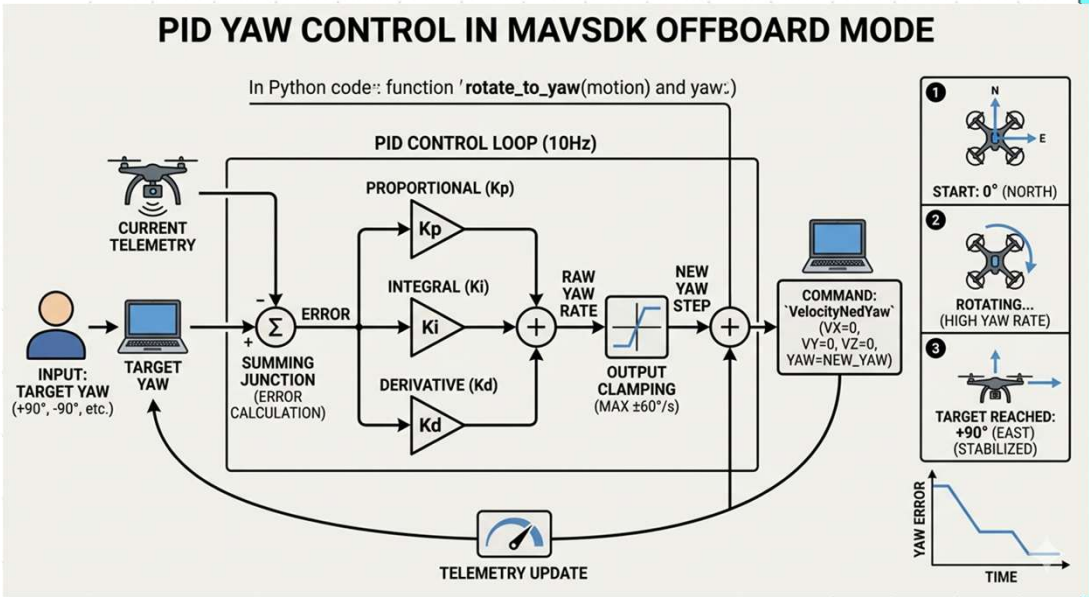
- Line 99 to line 167 in avoid.py implements the loop pipeline.
 - Line 117: Get position update.
 - Line 119: It starts by grabbing depthmap array using depth_receiver's get_frame()
 - Line 124: Calls AvoidancePlanner compute_position_ned to get next move.
- Besides command the PX4 to fly, the loop attempts to implement some high level decision (line 139 to line 143) such as rotating right or left if the front is completely blocked.
 - This is where your team implements the search strategy.**
 - The next workshop will cover this aspect and mapping which using histogram to generate map.

```
96 #
97 # THE MAIN LOOP WHERE THE PIPELINE COMES TOGETHER
98 #
99 async def run(self):
100     print("\nPOSITION-BASED AUTONOMOUS AvoidanceNAVIGATION\n")
101
102     await self.drone.connect()
103     await asyncio.sleep(3)
104     print("Starting position monitor.")
105     self.monitor_task = asyncio.create_task(position_monitor_task(self.drone, self.position_state, asyncio.Event()))
106     await self.drone.arm_and_takeoff()
107     # Initial alignment
108     await self.drone.rotate_to_yaw(self.target_yaw_deg)
109
110     try:
111         while self.running:
112             t_start = time.monotonic()
113
114             # -----
115             # UPDATE POSE (CRITICAL)
116             # -----
117             await self.update_pose()
118
119             depth_frame = self.receiver.get_frame()
120
121             # -----
122             # POSITION PLANNER
123             # -----
124             north, east, down, info = self.planner.compute_position_ned(
125                 depth_frame,
126                 self.pose,
127                 step_size=1.5
128             )
129
130             c = info['clearance']
131
132             print(f"Blocked={info['blocked']} | "
133                   f"Target N={north:.2f}, E={east:.2f} | "
134                   f"L={c['left']:.2f} C={c['center']:.2f} R={c['right']:.2f}")
135
136             # -----
137             # BLOCK HANDLING
138             # -----
139             if info['blocked']:
140                 await self.drone.send_velocity(0, 0, 0, self.target_yaw_deg)
141                 await self.rotate_next_direction()
142             else:
143                 # Ensure alignment before motion
144                 await self.align_to_grid()
145
146                 # -----
147                 # SEND POSITION SETPOINT
148                 # -----
149                 await self.drone.send_position_setpoint(
150                     north=north,
151                     east=east,
152                     down=down,
153                     yaw_deg=self.target_yaw_deg
154                 )
```

PLEASE refer to `vel_avoidance.py` while reading this slide

Using `drone_control.py` to command PX4

- `avoid.py` uses `Drone` object in `drone_control.py` which wrap MAVSDK functions into moer purposeful function call for ease of use.
- Core Class (`Drone`):** This encapsulates all drone communication and control functions. It initializes the `mavsdk.System()` object.
 - Connection and Setup:**
 - `connect()` : Establishes the UDP connection to the drone simulator.
 - `arm_and_takeoff()` : Handles arming, taking off, and crucially, pre-loads a 'zero velocity' command before activating **Offboard Mode**.
 - Basic Operations:**
 - `land()` : Stops offboard mode and triggers landing.
 - `get_yaw()` : Fetches the current heading in degrees from telemetry.
 - The Core Control Logic (PID Rotation):**
 - `rotate_to_yaw(target_yaw_deg)` : This is the most complex function. It implements a **PID (Proportional-Integral-Derivative)** control loop to smoothly rotate the drone to a specific heading while keeping its position stable (zero horizontal/vertical velocity). It calculates the `_yaw_error`, applies the PID gains (`Kp`, `Ki`, `Kd`), clamps the output (max yaw rate), and iteratively updates the `VelocityNedYaw` command until the tolerance is met.
 - High-Level Rotation Commands:**
 - `turn_cw_90()`, `turn_ccw_90()`, `turn_cw_180()` : These are helper functions that read the current heading and call `rotate_to_yaw()` with an offset (e.g., +90, -90, or +180).

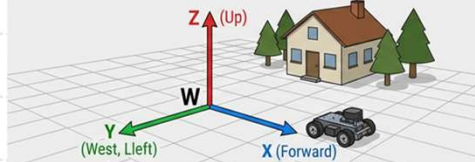


Differences in Coordinate Frame of drone, camera, and world

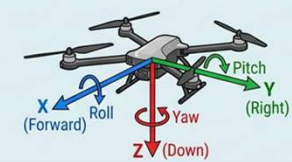
- Due to the differences on the facing x-y-z between camera data and PX4 commands, there is to "convert" or transform the x-y-z data before using to instruct PX4 or later using PX4 data for processing.
- As the drone rotates (yaw), the transformation is needed to ensure correctness of the coordinates.
- Thus, the Line 220 to line 222 in AvoidancePlanner.py function performs as a series of matrix rotation with respect to yaw facing of the quadcopter.
 - These are also known more in Supplementary Material.

DIFFERENCES IN COORDINATE FRAMES & DATA MAPPING

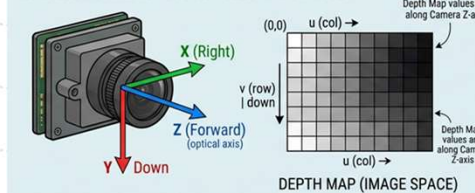
GAZEBO WORLD FRAME (NWU)



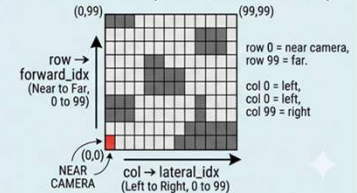
PX4/DRONE FRAME (FRD)



CAMERA & DEPTH CAMERA FRAME



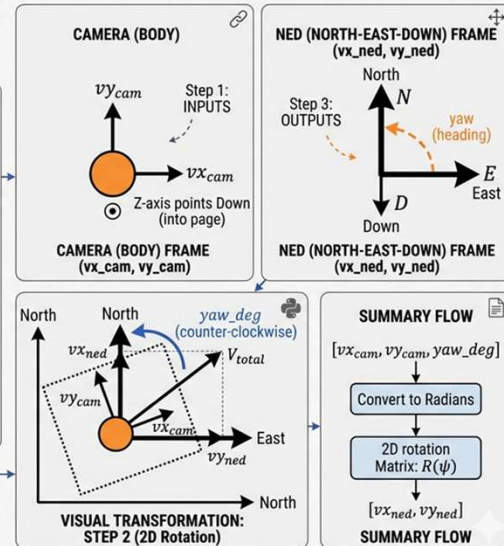
GRID MAPPING (OCCUPANCY GRID)



EXPLAINING `camera\_to\_ned(vx\_cam, vy\_cam, yaw\_deg)`: A COORDINATE FRAME TRANSFORMATION

```

220 # Rotation: body → NED
221 north_dir = vx_body * math.cos(yaw) - vy_body * math.sin(yaw)
222 east_dir = vx_body * math.sin(yaw) + vy_body * math.cos(yaw)
223
224 # Normalize direction
225 norm = math.sqrt(north_dir**2 + east_dir**2) + 1e-6
226 north_dir /= norm
227 east_dir /= norm
    
```



What's next for your team?

- Develop your code to avoid obstacles and navigate the roboverse.
 - Consider your high level strategy to handle situation when the quadcopter faces a blocked or even a deadend, and how can the histogram be used for mapping.
- You can develop your own or based it off the `avoid.py`, `drone_control.py`, `AvoidancePlanner.py` and `depth_receiver.py`, and `get_position_with_task.py`.
- Feel free to discuss and check out with our experts via the discord channel.

Codes that are discussed and their abilities

- These codes are discussed today and are available for download at RoboVerse Discord server's Learning Material Channel
 - `avoid.py`
 - Purpose: Glue other modules to implement the obstacle avoidance pipeline.
 - `drone_control.py`
 - Purpose: Abstract mavsdk into higher order function like `set_position` which is go to position and etc.
 - `depth_receiver.py`
 - Purpose: Interface with gazebo to get depth camera image.
 - `AvoidancePlanner.py`
 - Purpose: Process depth image and provide position to move.
 - `depthtest.py`
 - Purpose: Just to test depth camera.
 - `get_position_with_task.py`
 - Purpose: Get position and yaw position of drone in background

THANK YOU